

# SpectrumX — Documentation

Srinivaas A/L Thiruch Selvam | FSKTM, Universiti Malaya | Shortcut Asia Challenge 2026

## The Problem

I use Spectrum every day. It's where all my course materials, deadlines, and announcements live. But honestly, it's frustrating to use. Deadlines are scattered across 7 different course pages. Finding a single PDF means clicking through nested sections. Forum threads have 30+ replies and I never know what the actual takeaway is. And I keep forgetting to mark attendance because the button is buried 3 clicks deep.

There's already UMScout — but that's a separate app. You have to leave Spectrum, open another tab, sometimes re-enter info. I wanted something that just works where I already am.

## My Approach

I built SpectrumX as a Chrome extension that injects directly into Spectrum's pages. The idea is simple: don't make the student go somewhere else, make Spectrum itself better.

I started by looking at what actually annoys me day-to-day, talked to a few coursemates, and narrowed down to features that solve real friction. I focused on two core things first (deadline scanning + a smart dashboard), got those working, then layered on more features one at a time.

## Tech Stack

| What                           | Why I Picked It                                                                       |
|--------------------------------|---------------------------------------------------------------------------------------|
| Chrome Extension (Manifest V3) | Shares Spectrum's login cookies — no re-auth needed. Lives inside the browser.        |
| Vanilla JS + ES Modules        | No build step. I wanted to move fast without fighting Webpack configs.                |
| Chrome Offscreen Document API  | MV3 service workers can't parse HTML. Offscreen gives me DOMParser in the background. |
| Z.AI GLM-5.1 (for AI features) | OpenAI-compatible API. I had credits. Works well for structured JSON output.          |
| PDF.js                         | Client-side PDF reading. Needed it to extract text from assignment briefs.            |

## Decisions I Made (and Why)

**Extension instead of a web app.** Spectrum uses UM's SSO login. If I built a standalone site, users would have to deal with auth tokens or paste cookies. An extension just piggybacks on the existing session. Way simpler for the user.

**Background scanning via Offscreen Document.** Chrome's MV3 service workers don't have access to DOMParser — which I need to parse Moodle's HTML. I learned about the Offscreen Document API which lets me create a hidden page with full DOM access. The service worker tells the offscreen doc "go fetch this URL and parse it" and gets back structured data. Took me a while to figure out the messaging pattern but it works reliably now.

**Storage as the single data source.** Initially I tried scraping data from whatever Spectrum page the user was on. Bad idea — each page only shows a fragment. The dashboard has a timeline, course pages have activities, the calendar has events. Scraping the current page gave inconsistent results depending on WHERE you were. So I switched to having one "DeepScan" that fetches everything in the background and writes to chrome.storage. Every part of the extension reads from that one store.

**9 offline features + 3 AI features.** I wanted the core to work without any API key. AI is opt-in for the harder problems (reading PDFs, summarizing threads). If the Z.AI credits run out, the extension still works.

## How DeepScan Works

This is the main feature — it finds all your deadlines automatically. Here's the flow:

1. User clicks "Deep Scan" in the popup.
2. background.js creates a DeepScanner and opens the offscreen document.
3. Offscreen fetches the Spectrum home page, extracts all 7 enrolled courses from the course card elements.
4. Offscreen fetches the calendar month view, extracts events from each day cell (using the data-day-timestamp attribute for dates).
5. Offscreen fetches each course page in parallel, looks for assignment/quiz/workshop activities and tries to parse dates from the activity names.
6. Everything gets merged, deduplicated (same event from calendar + course page = one entry), and saved to chrome.storage.
7. The popup reads from storage and renders the dashboard with countdown, filters, and cards.

## How Forum TL;DR Works

1. Content script detects a forum discussion page (URL contains /mod/forum/discuss.php).
2. Injects a "Summarize" button below the title.
3. On click: extracts every post (author + content) from the page DOM.
4. Sends them to Z.AI with a prompt that asks for JSON: {tldr, consensus, actionItems, questions, tone}.
5. Renders a summary card that slides in from the right.

## Features Overview

| Feature           | What It Does                                   | Needs AI? |
|-------------------|------------------------------------------------|-----------|
| DeepScan          | Auto-finds all deadlines across all courses    | No        |
| Smart Dashboard   | Countdown timer, filters, urgency grouping     | No        |
| Spotlight (Cmd+K) | Search deadlines, courses, pages from anywhere | No        |
| Reader Mode       | Strips Moodle UI for clean reading             | No        |
| Focus Mode        | Hide all sections except one                   | No        |
| File Manager      | All files from all courses in one grid         | No        |
| Auto-Attendance   | Detect + mark open attendance sessions         | No        |
| Semester Export   | One-click course to markdown                   | No        |
| Forum TL;DR       | AI-summarizes forum threads                    | Yes       |
| Smart PDF Scan    | AI reads PDFs to find deadlines                | Yes       |
| AI Chatbot        | Ask about your deadlines in chat               | Yes       |

## What I'd Do Differently / With More Time

**Use Moodle's Web Services API.** Right now I scrape rendered HTML which is fragile — if UM changes their theme, selectors break. Moodle has a proper API (core\_calendar\_get\_calendar\_events, mod\_assign\_get\_assignments) that returns JSON. I didn't use it because I wasn't sure if UM's Spectrum exposes those endpoints, and I ran out of time to test. But that would be the right move for production.

**Better error handling in the attendance module.** The attendance feature is the newest and least tested. UM's attendance plugin has variations per-lecturer (some use passwords, some don't). I'd want to test across more courses before trusting it fully.

**Grade Calculator.** I scoped out a "What-If" grade calculator that overlays on the grades page — type hypothetical scores and see your projected final grade. Had to cut it for time, but the Moodle grades page has a clean table structure that would be straightforward to parse.